

Policy Versus Mechanism

Separating who decides from who enforces

Definition

Policy versus mechanism is the design principle that the operating system should provide general scheduling mechanisms, while policy choices can be supplied by the entity that has better knowledge of what should happen.

1. Old Assumption

Independent Competitors

Many scheduling discussions assume that all processes belong to different users and are competing independently for the CPU.

Often True

This assumption works for many time-sharing systems, where unrelated users and unrelated processes compete for processor time.

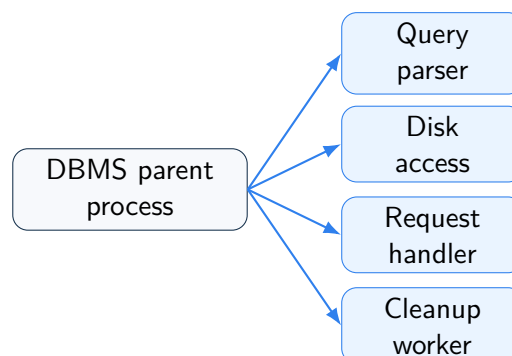
2. When the Assumption Fails

Related Processes

Sometimes one process creates and controls many child processes. These children are not independent competitors in the usual sense; they are parts of a larger application.

Database System

A database-management-system process may have many child processes. One child may parse queries, another may access the disk, another may handle client requests, and another may perform cleanup work.



3. The Knowledge Problem

Parent Knows More

The parent process may know which child is most important, which one is time critical, and which one can safely wait.

Kernel Knows Less

A general-purpose kernel scheduler often sees only runnable processes, priorities, CPU usage, and blocking behavior. It may not understand the application's internal priorities.

Result

If the scheduler accepts no useful input from user processes, it may choose a process that is technically runnable but not the best choice for the application.

4. Policy and Mechanism

Mechanism

A **mechanism** is the low-level facility that performs an action. In scheduling, the mechanism is the code that chooses and dispatches a process.

Policy

A **policy** is the rule that decides what should be done. In scheduling, the policy decides which process should be favored and why.

Concept	Question Answered	Example
Mechanism	How can scheduling be carried out?	Kernel priority scheduler.
Policy	Which process should receive better treatment?	Parent gives urgent child a higher priority.

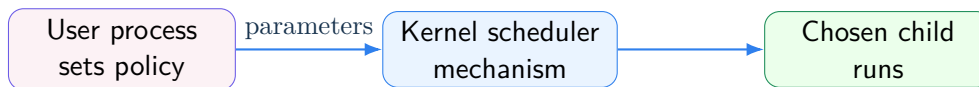
5. Separation Principle

Main Principle

Separate the scheduling mechanism from the scheduling policy. The kernel provides a parameterized scheduling mechanism, and user processes can supply or adjust the parameters.

Why It Helps

The kernel keeps control over the actual scheduling operation, while the application can express information that the kernel would not otherwise know.



6. Parameterized Scheduling

Parameterized Algorithm

A scheduling algorithm is **parameterized** when its behavior depends on adjustable values, such as priorities, weights, CPU shares, or deadlines.

Policy Input

The user process does not run the scheduler directly. It only supplies parameters that influence how the scheduler behaves.

7. Database Example

Kernel Mechanism, User Policy

Suppose the kernel uses priority scheduling and provides a system call that lets a parent process set or change the priorities of its children.

Part	Role
Kernel scheduler	Mechanism: dispatches runnable processes according to priority.
System call	Interface: lets a parent set child priorities.
DBMS parent process	Policy: decides which child is most important right now.
Child processes	Work units: query parsing, disk access, request handling, cleanup.

```

parent process observes its children

if a child is time critical:
    set that child's priority high
else if a child can wait:
    set that child's priority low

kernel scheduler:
    run runnable processes according to priority
  
```

8. What the Parent Controls

Control Without Direct Scheduling

The parent controls how its children are likely to be scheduled by changing their scheduling parameters. It does not directly take over the CPU scheduler.

Kernel Still Enforces

The kernel still performs the actual scheduling decision, protects the system, and enforces rules about what a user process is allowed to change.

9. Why Not Put All Policy in the Kernel

Too Much Application Knowledge

The kernel cannot reasonably know every application's internal meaning of importance, urgency, and dependency.

Better Location

Application-specific policy often belongs near the application, because the application understands its own children and their roles.

10. Security and Limits

Need for Constraints

Letting user processes influence scheduling must be controlled. Otherwise a process could unfairly raise all of its children above other users' processes.

Safe Design

The kernel can restrict policy changes to a process's own children, limit priority ranges, enforce permissions, and still apply global fairness rules.

11. Advantages

Advantage	Explanation
Flexibility	Different applications can express different scheduling needs.
Better decisions	The application can provide information the kernel does not know.
Cleaner design	The kernel provides general mechanisms instead of hard-coding every policy.
Controlled power	The kernel can allow useful influence without giving up protection.

12. Comparison

Design	How Scheduling Is Decided	Main Weakness
Fixed kernel policy	Kernel makes all policy decisions internally.	May ignore application-specific knowledge.
Policy-mechanism separation	Kernel provides mechanism; user process supplies parameters.	Requires careful interfaces and protection.

13. Big Picture

Main Conclusion

Policy-mechanism separation lets the kernel keep control of scheduling while allowing user processes to express application-specific scheduling policy. The mechanism stays in the kernel; the policy can be influenced by the process that understands the work.

Exam Shortcut

Remember: mechanism means how scheduling is performed; policy means what scheduling choice is desired. In the database example, the kernel priority scheduler is the mechanism, and the parent process setting child priorities is the policy.

14. Quick Review

Key Points

- Many schedulers assume processes are independent CPU competitors.
- Sometimes one parent process controls many related child processes.
- A DBMS may have children for query parsing, disk access, request handling, and other work.
- The parent may know which child is most urgent or important.
- The kernel scheduler may not know the application's internal priorities.
- Policy-mechanism separation solves this by keeping the scheduler mechanism general.
- The kernel provides the scheduling mechanism.
- The user process can supply policy by setting scheduling parameters.
- Example: a parent uses a system call to change child priorities.
- The parent influences scheduling, but the kernel still performs and controls scheduling.
- The kernel must enforce permissions and limits to prevent abuse.